

Southwest
Tech

Python Programming Foundations

Week 3 - Organizing Code and Data
Day 2: "Storing Related Data w/Lists & Dictionaries"

1

Programs Become Useful When They Remember Things

Programs become more useful when they can keep related values together.

Lists and dictionaries help a program remember, retrieve, and use information.

Today, watch for:

- ▶ what data is stored
- ▶ how the data is accessed
- ▶ how the program uses the data
- ▶ which structure fits the task

2

Southwest
Tech

Small Program, Related Values, Useful Output

Group related values. Use them together. Produce something useful.

Related Values (In Memory)

- tasks
 - attend class
 - complete assignment
 - study for test
 - read chapter
- scores
 - homework 1: 88
 - Quiz 1: 76
 - Midterm: 92
 - Final: 85
- Course Details
 - Course Intro to Python
 - Student: Alex
 - Term: Spring 2025

Python Program

```

# related values
tasks = ["attend class", "complete assignment",
        "study for test", "read chapter"]
scores = {"homework 1": 88, "Quiz 1": 76,
          "Midterm": 92, "Final": 85}
course = {"name": "Intro to Python",
          "student": "Alex", "term": "Spring 2025"}

# calculations
total = sum(scores.values())
average = total / len(scores)
highest = max(scores.values())
lowest = min(scores.values())

# output
print(f"Course: {course['name']}")
print(f"Student: {course['student']}")
print(f"Term: {course['term']}")
print(f"Tasks: {tasks}")
print(f"Scores: {scores}")
print(f"Average Score: {average}")
print(f"Highest Score: {highest}")
print(f"Lowest Score: {lowest}")

```

Useful Output

```

Course: Intro to Python
Student: Alex
Term: Spring 2025

Task List:
1. attend class
2. complete assignment
3. study for test
4. read chapter

Scores:
Homework 1: 88
Quiz 1: 76
Midterm: 92
Final: 85

Average Score: 85.3
Highest Score: Midterm (92)
Lowest Score: Quiz 1 (76)

```

Group related values. Process them together. Produce useful information. This is the foundation of every program.

Related Values

3

Southwest
Tech

Logic Uses Data

Functions organize logic.
Lists and dictionaries organize information.

▶ Small programs often need both:

- ▶ a function for the job
- ▶ a data structure for the values
- ▶ output that shows the result

4

Southwest
Tech

Function Uses Data to Produce a Result

logic + related data → visible result

logic

```

def calculate_average(scores):
    """Find the average of the scores.

    Steps (Hidden Inside the Function):
    1. Add up all the scores.
    2. Count how many scores there are.
    3. Divide total by count.
    4. Return the average.
    """

```

related data

scores (list)

```

[88, 76, 92, 85]

```

OR

student (dictionary)

```

{
    "name": "Alex",
    "scores": [88, 76, 92, 85],
    "term": "Spring 2025"
}

```

visible result

Average Score: 85.25

Based on the scores: [88, 76, 92, 85]

Logic does the work. Related data provides the values. A visible result is produced. Change the data, get a new result.

Logic Plus Data

5

What Counts as Success Today

Store related data -> retrieve useful values -> loop or look up -> show output.

For each data structure, ask:

What values are stored?	How do I access one value?	How do I use all values?	What visible behavior proves it works?
-------------------------	----------------------------	--------------------------	--

6

Southwest Tech

From Data to Output: A Simple 4-Step Flow

Small program. Clear steps. Useful result.

1 **Store related data**
Keep related values together in a list or dictionary.
`scores [int]`
`[88, 76, 92, 85]`

2 **Retrieve useful values**
Get the values you need from the data.
`total = 341`
`count = 4`
`average = 85.25`

3 **Loop or look up**
Loop through items or look up information as needed.
Processing scores...
`88 = added`
`76 = added`
`92 = added`
`85 = added`

4 **Show output**
Present the result in a clear, readable way.
`Results`
`Total: 341`
`Count: 4`
`Average: 85.25`

Good programs follow clear steps from data to output. Change the data, run again, get a new result.

Today's Success

7

Southwest Tech

What We Will Use Today

- ▶ lists
- ▶ indexes
- ▶ `'append()'`
- ▶ loops over lists
- ▶ dictionaries
- ▶ keys and values
- ▶ dictionary lookup

These tools help a program work with more than one related value.

```

mirror_mod = modifier_obj
mirror_mod.mirror_object =
mirror_mod.mirror_object =
operation = "MIRROR_X"
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
operation = "MIRROR_Y"
mirror_mod.use_x = True
mirror_mod.use_y = True
mirror_mod.use_z = False
operation = "MIRROR_Z"
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

selection at the end - add
obj.select = 1
for obj.select in
context.scene.objects.active
("Selected" + str(modifier
mirror_obj.select = 0
bpy.context.selected_obj
data.objects[one.name].sel
int("please select exactly
-- OPERATOR CLASSES --

types.Operator):
X mirror to the select
object.mirror_mirror_x"
mirror X"

context):
next-active_object is ne

```

8

Southwest Tech

Python Data Toolbox

Simple concepts. Powerful tools.

List
Ordered collection of items.
`items = ["apple", "banana", "carrot"]`

Index
Position of an item in a list (starts at 0).
`items[0] # "apple"`
`items[-1] # "carrot"`

append
Add an item to the end of a list.
`items.append("data")`
`# ["apple", "banana", "carrot", "data"]`

loop
Repeat actions for each item in a list.
`for item in items:`
`print(item)`

dictionary
Collection of key-value pairs.
`person = {"name": "Bob", "age": 20}`

key
Unique label used to find a value.
`"name" # key in person`

value
The data associated with a key.
`person["name"] # "Bob"`

lookup
Use a key to find the related value.
`person["age"] # 20`

These tools work together to store, find, and use data. Practice with small examples. Build confidence step by step.

Toolbox

9

Southwest Tech

What We Will "Skip" For Now

- ▶ Comprehensions
- ▶ Generators
- ▶ Advanced collections
- ▶ Databases
- ▶ Complex nested data

10

Southwest Tech

Parked for Later

Useful later, not today's focus.

comprehensions
Compact ways to create lists, sets, or dictionaries.
`squares = [x**2 for x in range(10)]`

generators
Produce values one at a time on demand.
`def count():`
 `n = 1`
 `while True:`
 `yield n`

advanced collections
Specialized containers for specific tasks.
`from collections import deque, defaultdict`

databases
Store and query large amounts of persistent data.
`SELECT * FROM students WHERE grade > 90;`

complex nested data
Deeply nested structures and relationships.
`{ "user": { "name": "Alice", "tasks": [ ... ] }}`

We'll come back to these when you're ready.

Master the basics first. These topics build on that foundation. You'll use them at the right time.

Parked For Later

11

Southwest Tech

Lists Keep Related Items Together

A list stores multiple related values in one variable.

Example:

```

Code_Sample.py
1
2 tasks = ["read", "practice", "submit"]
3

```

Use a list when order or repeated display matters.

12

A List of Tasks in Python

A list groups related items together in order.

one list variable

```
tasks = [
    [read icon, read text: Read the chapter for class.],
    [practice icon, practice text: Practice the exercises.],
    [submit icon, submit text: Submit the assignment before the deadline.]
]
```

★ The variable `tasks` holds a list with three items: `read`, `practice`, `submit`.

List Container

13

List Positions Start At Zero

List items are accessed by position.

Python starts counting positions at '0'.

Example:

```
1 tasks = ["read", "practice", "submit"]
2
3
4 value = tasks[0] ←
5
```

This gets the first task.

14

Python List Indexing (Zero-Based)

Indices start at 0.

```
tasks = ["read", "practice", "submit"]
```

first item →

Index 0	Index 1	Index 2
read	practice	submit
index 0	index 1	index 2

★ Zero-based indexing means the first item is at index 0.
Index 0 → first, Index 1 → second, Index 2 → third.

Zero-Based Index

15

Demo #1

"List Basics"

16

What to Watch For

Watch the list change.

Trace:

- the starting values
- the item accessed by position
- the loop that displays all items
- the new item added with `append()`

17

Lists Grow with `append()`

Add an item to the end of a list.

BEFORE tasks (3 items)

```
tasks = [read, practice, submit]
```

APPEND ACTION
`tasks.append("review")`
A new item is added to the end.

AFTER tasks (4 items)

```
tasks = [read, practice, submit, review]
```

LOOP OUTPUT
Looping through the list shows each task in order:
1. read
2. practice
3. submit
4. review

Lists keep items in order. `append()` adds to the end.

List Demo

18

Looping Makes A List Useful

A list becomes useful when the program can use each item.

Common beginner pattern:

```
Code_Sample.py > ...
1
2 tasks = ["read", "practice", "submit"]
3
4 for task in tasks:
5     print(task)
6
```

The loop handles one item at a time.

19

Dictionaries

20

Dictionaries Connect Meaning To Value

A dictionary stores values by key.

Example:

```
Code_Sample.py > ...
1
2 course = {"title": "Python Programming", "credits": 2}
3
```

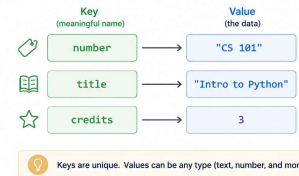
The key is a meaningful label for the value.

21

Python Dictionary: Key / Value Pairs

A dictionary stores related data using meaningful keys.

```
course = { "number": "CS 101", "title": "Intro to Python", "credits": 3 }
```



Dictionary Pairs

22

Keys Are Not Positions

- ▶ Lists use **positions**.
- ▶ Dictionaries use **keys**.

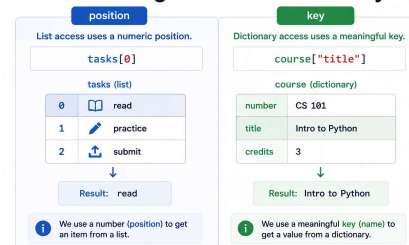
Compare:

```
Code_Sample.py > ...
1
2 tasks = ["read", "practice", "submit"]
3 course = {"title": "Python Programming", "credits": 2}
4
5 tasks[0] # Accessing the first task
6 course["title"] # Accessing the course title
7
```

The list asks for a position. The dictionary asks for a label.

23

Accessing Data: Position vs Key



Position vs. Key

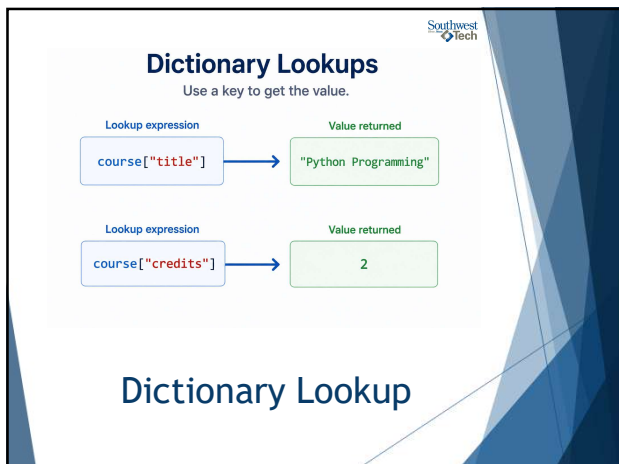
24



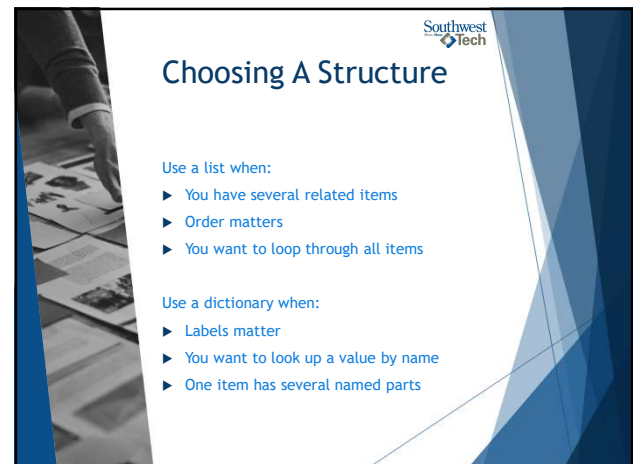
25



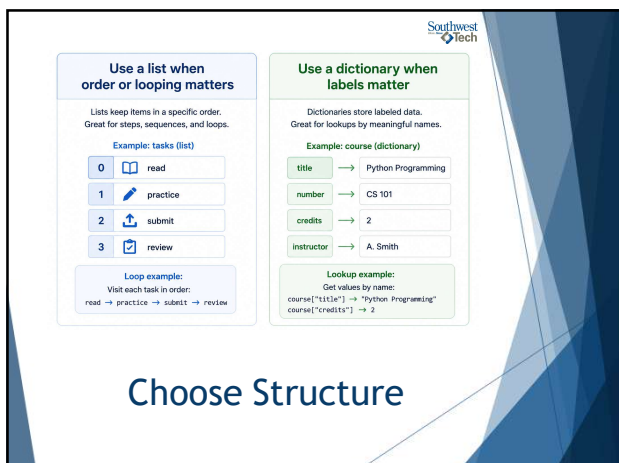
26



27



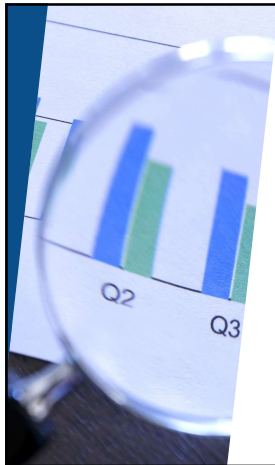
28



29



30



What to Watch For

A list of dictionaries can store multiple records.

In the demo, each student has:

- ▶ a name
- ▶ a score
- ▶ a status decision

The loop handles one record at a time.

31



Lists Can Hold Structured Records

A list can hold many small, related records.

students (list of records)

Record 0	Record 1	Record 2
name: Maya	name: Jordan	name: Luis
score: 92	score: 85	score: 78

Loop idea: Visit one record at a time.

This is a bridge to future structured data. You don't need this right now, but lists of records are a powerful next step.

Demo 3: List of Dictionaries

32



Assignment #5

Build one small mini-app using a list or dictionary.

A good first version should include:

- ▶ one chosen structure
- ▶ a few sample values
- ▶ one useful lookup or loop
- ▶ visible output
- ▶ a short explanation of why the structure fits

33



Mini-App Ideas: Pick Any One

Different apps, different data structures, visible results.

Task List	Simple Inventory	Score Tracker	Course Lookup
Manage your tasks	Track items and counts	Record and view scores	Look up course info
Data structure: list	Data structure: dictionary	Data structure: list of records	Data structure: dictionary
["read", "practice", "submit"]	{ "apple": 4, "book": 2, "bottle": 1 }	[{"name": "Maya", "score": 92}, {"name": "Jordan", "score": 85}, {"name": "Luis", "score": 78}]	{ "title": "Python 101", "credits": 2, "instructor": "A. Smith" }
Visible output: My Tasks	Visible output: Item Count	Visible output: Name Score	Visible output: Title
read practice submit	apple: 4 book: 2 bottle: 1	Maya: 92 Jordan: 85 Luis: 78	Python 101 Credits: 2 Instructor: A. Smith

Each mini-app helps you practice core ideas. Choose the one that interests you most.

All are great ways to learn. None is required.

Assignment #5

34



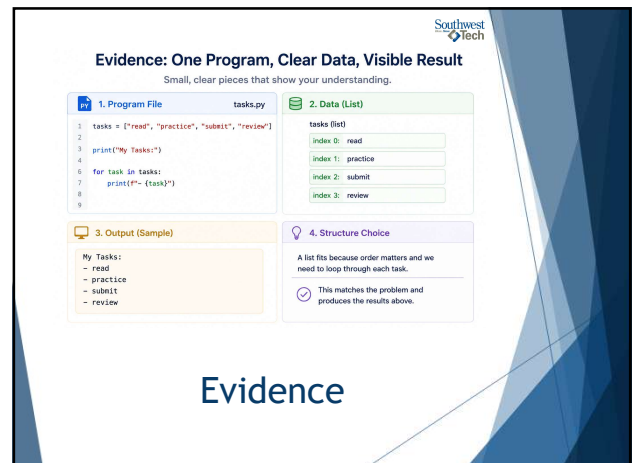
Evidence for A Data Mini-App

Your submission should show that the data structure is useful.

Useful evidence includes:

- ▶ one working '.py' file
- ▶ list or dictionary data
- ▶ retrieval, lookup, or loop behavior
- ▶ visible output
- ▶ short explanation of the structure choice

35



Evidence: One Program, Clear Data, Visible Result

Small, clear pieces that show your understanding.

1. Program File (tasks.py)

```
1 tasks = ["read", "practice", "submit", "review"]
2
3 print("My tasks:")
4
5 for task in tasks:
6     print(f"-- {task}")
7
8
9
```

2. Data (List)

```
tasks (list)
index 0: read
index 1: practice
index 2: submit
index 3: review
```

3. Output (Sample)

```
My Tasks:
-- read
-- practice
-- submit
-- review
```

4. Structure Choice

A list fits because order matters and we need to loop through each task.

This matches the problem and produces the results above.

Evidence

36



37